DESIGN, IMPLEMENTATION AND SCALABILITY ANALYSIS OF AN AUTOMATED DESKTOP-GRID HPC CLUSTER USING IDLE ACADEMIC COMPUTER LABS

PROJECT PROPOSAL REPORT

CO3563 High Performance Computing Project

NAME	: B. A. B. N. Ariyarathna
REGISTRATION NO	: EN113753
INDEX NO	: 23/ENG/017
SUPERVISOR	: Dr. Chinthaka Kumara
SUBMISSION DATE	: 21/03/2026

1. INTRODUCTION

Engineering simulations, finite element methods and parameter tuning for machine learning applications require more computational resources than a typical student workstation can provide. Dedicated clusters and cloud computing are available, but their usage fees can be high, especially for student projects.

Computer laboratories contain a large number of multi-core desktop PCs, but these are used mainly during teaching hours and are expected to sit idle at other times, while researchers wait for computing resources. This project will measure how much idle time is actually available on the laboratory computers.

These computers can be connected into a Beowulf-style cluster over Gigabit Ethernet, but its latency is much higher than that of the specialized interconnects used in supercomputers. As a result, tightly coupled MPI programs may scale poorly, and quantifying this is one of the aims of the project.

This project designs, implements and evaluates a desktop-grid system based on the Slurm workload manager [2], Docker containers [4] and an idle-detection daemon. Idle computers are added to the cluster automatically, and a computer is released as soon as a user sits down at it.

2. PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Transforming shared PCs in laboratories into an effective computation cluster poses four major difficulties:

1. **Untapped idleness.** Lab PCs are expected to be idle for a large part of the week, but their actual usage has not been measured. The project will record it.

2. **Slow interconnect.** A 1 Gbps Ethernet switch has much higher latency and lower bandwidth than supercomputer interconnects, so communication-heavy MPI applications may scale poorly.

3. **Unstable, non-dedicated nodes.** A student may sit down at any PC at any time. An MPI job is not fault tolerant, so losing one node causes the whole job to fail.

4. **Host stability.** Lab computers must stay clean and stable for teaching. Installing compilers and scientific libraries on every computer risks breaking them and is time-consuming to manage.

2.2 Objectives

The goal is to create, deploy and evaluate an automated and containerized desktop-grid HPC system on unused laboratory computers. The specific, quantifiable goals include:

1. **Cluster architecture:** Build a master–worker cluster with one head node and up to 16 worker nodes, using NFS shared storage and MUNGE authentication over Gigabit Ethernet.

2. **Orchestration:** Set up Slurm to enable batch job scheduling and multi-node assignment and to allow for job requeueing and the ability to program the status of nodes.

3. **Containerized runtime:** Run the OpenMPI workload inside Docker containers utilizing host networking and assess the overhead in comparison with bare-metal.

4. **Idle-harvesting daemon:** Develop a Python daemon that enlists a node after 15 minutes of idleness and evicts all cluster work from it when a user returns. The target is to release the CPU within 1 second in at least 95% of trials.

5. **Scalability evaluation:** Benchmark HPL and selected NPB kernels on 1–16 nodes and report speedup, parallel efficiency and communication time with reference to Amdahl's Law [7].

3. LITERATURE REVIEW

3.1 Cycle Scavenging

Condor [1] showed that idle workstations can be used for production computing, with jobs suspended or moved when the owner returns. This model suits batch workloads made of many independent jobs. Tightly coupled MPI applications are harder to support, since losing one node fails the whole job.

3.2 Scheduling, Messaging and Containers

Slurm [2] is a widely used open-source scheduler for HPC clusters, with support for MPI job allocation, requeueing and node-state management. OpenMPI [3] provides the message-passing layer and can communicate over TCP on Ethernet. Docker [4] packages an application with its dependencies so that the same environment runs on every node. Container overhead is generally reported to be small for CPU-bound work, but this project will measure it on its own testbed rather than assume it.

3.3 Benchmarks

High-Performance Linpack [5] measures dense linear algebra performance in GFLOPS and is mainly compute-bound at large problem sizes. The NAS Parallel Benchmarks [6] consist of kernels with different communication patterns, which makes them suitable for exposing network limits.

3.4 Gap and Positioning

HTCondor already provides idle detection and preemption and is the closest existing alternative. This project uses Slurm instead because it is a standard scheduler for MPI jobs on HPC clusters and is likely to be met in later research work. The contribution is a lightweight, containerized layer that adds idle detection and safe eviction to Slurm, together with a measured evaluation of how far a Gigabit Ethernet desktop cluster can scale. For purely independent tasks, HTCondor may be the better choice, and the results will say so where relevant.

4. EXPERIMENTAL TESTBED & DESIGN

4.1 Hardware Testbed

The test bed is made up of a head node and as many as 16 worker PCs connected via a dedicated 24-port Gigabit switch, isolating test traffic from the school's network. To obtain pertinent speedup results, workers having similar specifications will be used for the scaling tests.

Table 1: Hardware Testbed

Component	Specification	Role
Head node	Quad-core 3.4 GHz CPU, 16 GB RAM, 512 GB SSD	Slurm controller, NFS server
Workers (up to 16)	Identical Intel Core i5/i7 PCs, 8–16 GB RAM (exact models to be recorded)	Run containerized MPI tasks and the idle-detection daemon
Network	24-port 1000BASE-T switch, Cat6 cabling	Isolated cluster network

4.2 Workloads

- **HPL:** double precision linear solver while reporting GFLOPS, and it is expected to be compute dominant.
- **NPB kernels:** comprise IS (integer sorting, random access, and communication), FT (3D FFT and all to all communication) and MG (multigrid with short messages).
- **Independent-task workload:** a Slurm job array of single-node tasks. This is the workload type that desktop grids suit best, and it is used to compare throughput with and without harvesting.

4.3 Metrics and Method

Node count is $P \in \{1, 2, 4, 8, 16\}$. Strong scaling (fixed problem size) is used for all benchmarks, and weak scaling is added for HPL. Each configuration is run at least five times and reported as mean $\pm$ standard deviation. With T_1 the single-node time and T_P the time on P nodes:

$$S(P) = \frac{T_1}{T_P}, \qquad E(P) = \frac{S(P)}{P}, \qquad e_{KF}(P) = \frac{1/S(P) - 1/P}{1 - 1/P} \tag{1}$$

where e_{KF} is the Karp–Flatt estimate of the effective serial fraction. Amdahl's Law [7] gives the speedup bound

$$S(P) \leq \frac{1}{f + (1 - f)/P}, \tag{2}$$

where f denotes the serial fraction. The execution of the e_{KF} indicates if the serial fraction is invariant or increases with P, due to communication overhead. The raw latency of the network and bandwidth are measured using `iperf3` tool and OSU micro-benchmarks and the communication time for each execution T_{comm} is profiled using MPI profilers such as mpiP. The overhead in Docker is determined by comparing running in containers against running on bare metal on the same systems. Eviction latency T_{evac} calculated as the time taken to run from input event until CPU release is measured over many repetitions and averaged along with 95th percentile and maximum values.

4.4 Hypotheses

- **H1:** HPL reaches at least 70% parallel efficiency on 8 nodes at large problem sizes, because computation dominates communication.
- **H2:** FT and IS show falling efficiency as P grows, because all-to-all traffic saturates 1 Gbps links.
- **H3:** Docker with host networking adds less than 5% overhead compared with bare metal.
- **H4:** The daemon frees the CPU within 1 second of user input in at least 95% of trials.

4.5 Storage

An NFS share on the head node, mounted at the same path on all workers, holds executables, inputs and logs. Benchmark scratch data is placed in RAM-backed storage (`tmpfs`) where possible to avoid disk and NFS bottlenecks. Metrics from monitoring scripts are written to CSV files on the share.

5. PROPOSED ARCHITECTURE

A researcher submits a batch script to the head node. The Slurm controller (`slurmctld`) assigns the job to available workers, where `slurmd` starts it inside a Docker container. Processes communicate over TCP using OpenMPI, and the same container image is used on every node. The cluster layout is shown in Figure 1.

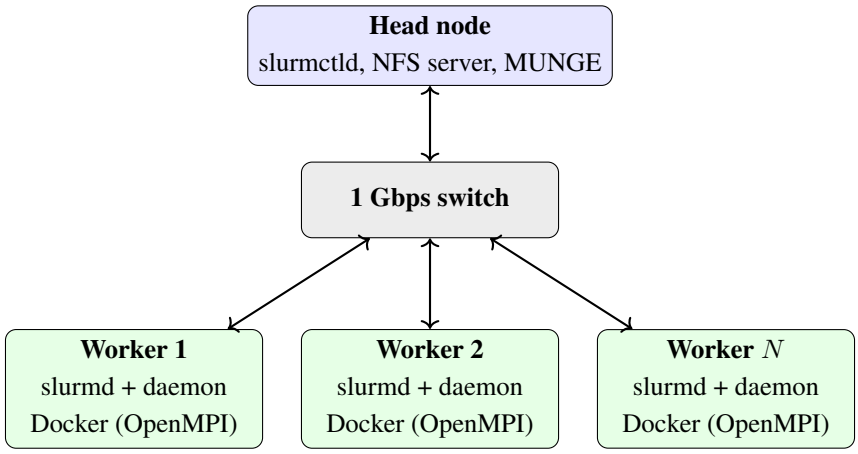


Figure 1: Cluster architecture.

Variant 1: Single-Node Baseline

The benchmarks run on one workstation without network distribution, providing T_1.

Variant 2: Distributed MPI Pipeline

The same benchmarks run on P nodes under OpenMPI. Each application uses its own data decomposition (for example, HPL uses a 2D block-cyclic distribution). This variant shows how well extra nodes reduce run time against Ethernet overhead, and is run on dedicated nodes without the daemon.

Variant 3: Automated Harvesting Pipeline

Variant 3 is the main engineering contribution. It adds the idle-harvesting daemon to Variant 2. The daemon's states are shown in Figure 2 and its eviction sequence in Figure 3.

- **Idle detection:** The daemon runs as a root `systemd` service. It reads input activity from `/dev/input/event*` using `evdev`, which works regardless of the display server. It also checks for active login sessions, so a logged-in user who is not touching the keyboard (for example over SSH) is not treated as idle, and it checks that CPU load is low before enlisting a node. Only the time of the last input event is kept, never key content.

- **Enlistment:** After 15 minutes without input, with no active login session and low CPU load, the daemon runs `scontrol update NodeName=<node> State=RESUME`, making the node available to Slurm. All workers are listed in `slurm.conf` and are kept in the `DRAIN` state while in use. Requiring 15 continuous minutes of idleness prevents nodes from repeatedly joining and leaving the pool.

- **Eviction:** On the first input event, the daemon (1) force-stops the local grid containers with `docker kill`, since `docker stop` waits 10 seconds by default, (2) sets the node to `DRAIN`, started in parallel so that no new work is placed on it, and (3) requeues the jobs running on the node with `scontrol requeue`, which makes Slurm terminate all tasks of those jobs. Containers on the other nodes of a requeued job are removed by a Slurm epilog script that stops containers labelled with the job ID, because stopping a Slurm step does not stop a container started through the Docker daemon. T_{evac} is measured until the containers are confirmed removed (Figure 3, step 2).

- **Limitation:** Slurm requeues whole jobs, not individual MPI ranks, so a tightly coupled job loses its progress and restarts from the beginning on the remaining nodes. Checkpointing is outside the project scope and listed as future work. Because of this, the independent-task workload is expected to benefit most from harvesting, and the evaluation will compare it with the MPI workloads.

- **Isolation:** Since workloads run inside containers, stopping them leaves no software or configuration changes on the host.

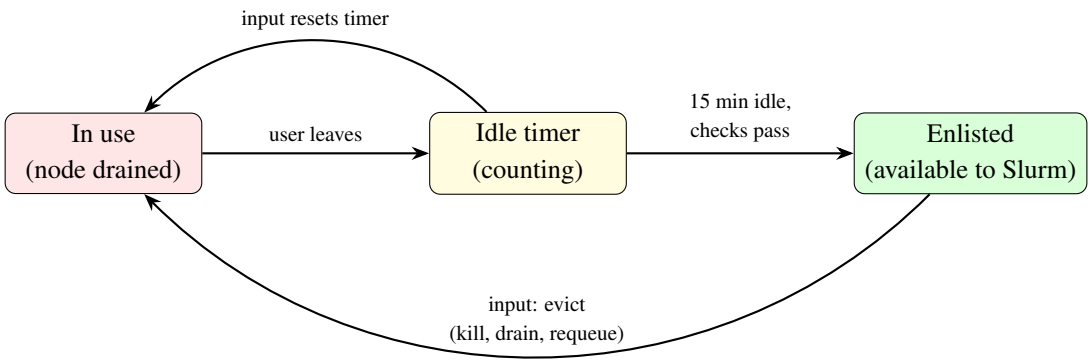


Figure 2: Idle-harvesting daemon states.

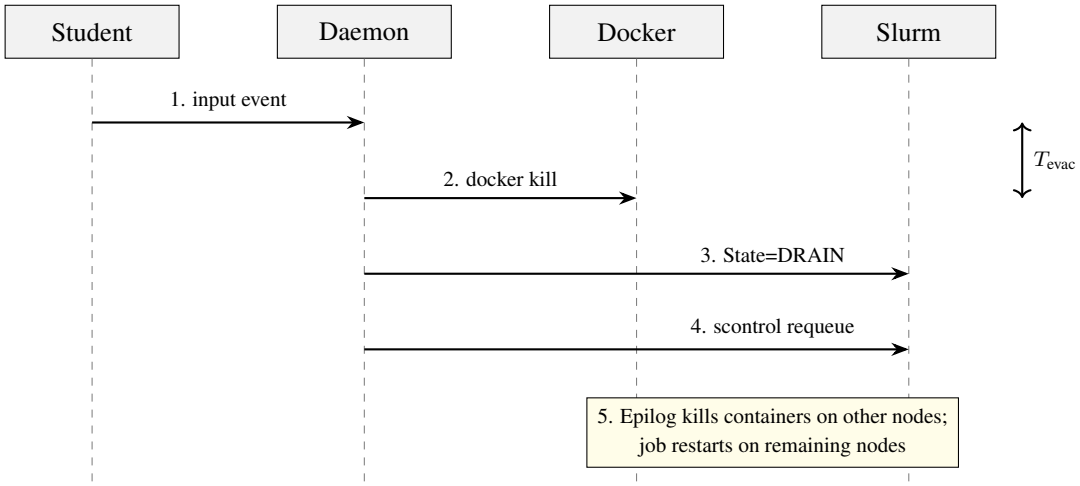


Figure 3: Eviction sequence when a user returns to a node. T_{evac} is measured from the input event to confirmed container removal.

6. TOOLS AND TECHNOLOGIES

Table 2: Technology stack

Tool	Purpose	Reason / alternative
Ubuntu 24.04 LTS	Operating system on all nodes	Long-term support, full kernel input access
Slurm + MUNGE	Scheduling, authentication	Standard HPC scheduler; alternative is HTCondor
Docker (host network)	Portable runtime	Familiar and widely supported. Apptainer or rootless Docker would reduce the security risk of root-equivalent Docker access and may be tested if time allows
OpenMPI	MPI communication	Common open-source MPI over TCP
Python 3 (`evdev`, `psutil`)	Idle-detection daemon	Rapid development and easy Slurm CLI integration
Ansible	Automated node configuration	Reproducible setup of many PCs
NFS	Shared storage	Simple, standard; limited by the 1 Gbps link

7. RISKS AND ETHICAL CONSIDERATIONS

Table 3: Main risks and mitigations

Risk	Mitigation
Head node failure (single point of failure for Slurm and NFS)	Regular backups of configuration; accepted as a limitation for this scale
Daemon crash or hung eviction	`systemd` auto-restart; watchdog that kills containers if the daemon is not running
Network contention from other lab traffic	Dedicated switch for benchmark runs
User interruption of long MPI jobs	Evaluate independent-task workloads; document restart-from-scratch behavior

Permission from the department and IT staff is required before installing software on lab machines. The daemon records only idle time, not keystrokes. Power use and machine wake-up or suspend behavior will be agreed with the lab administrators.

8. PROJECT TIMELINE

The project spans Semester 6 (15 weeks), a 6-month industrial internship (about 26 weeks), and Semester 7 (15 weeks). The internship is used to gain DevOps experience that will be applied to the Ansible automation; no benchmarking is planned during it.

Table 4: Phases and deliverables

Phase	**Tasks**	**Deliverable**
1. Proposal (Oct–Nov 2026)	Proposal report, presentation and viva; create the GitHub repository	Proposal accepted; repository shared with evaluators
2. Testbed (Sem 6)	Static IPs, SSH, NFS, MUNGE on 4 nodes; confirm lab OS version and access	Working 4-node testbed
3. Slurm and Docker (Sem 6)	Install Slurm; build OpenMPI image	Multi-node MPI job inside Docker
4. Daemon and first benchmarks (Sem 6)	Idle detection, enlistment, eviction; initial 4-node benchmarks	Nodes resume and drain automatically; first results
5. Industrial training (about 6 months)	DevOps practice; Ansible playbooks; no benchmarking planned	Automated deployment playbooks
6. Mid-term (9–29 Aug 2027)	Presentation, viva and 5-minute video	Progress since the proposal demonstrated
7. Scaling (Sem 7)	Scale to 16 nodes; HPL, NPB and throughput tests	Benchmark and eviction datasets
8. Paper and final evaluation (Sem 7)	Analysis against Amdahl's Law; ICML-format paper and peer reviews; final presentation, demo and viva (13 Dec 2027–2 Jan 2028)	Final paper and viva

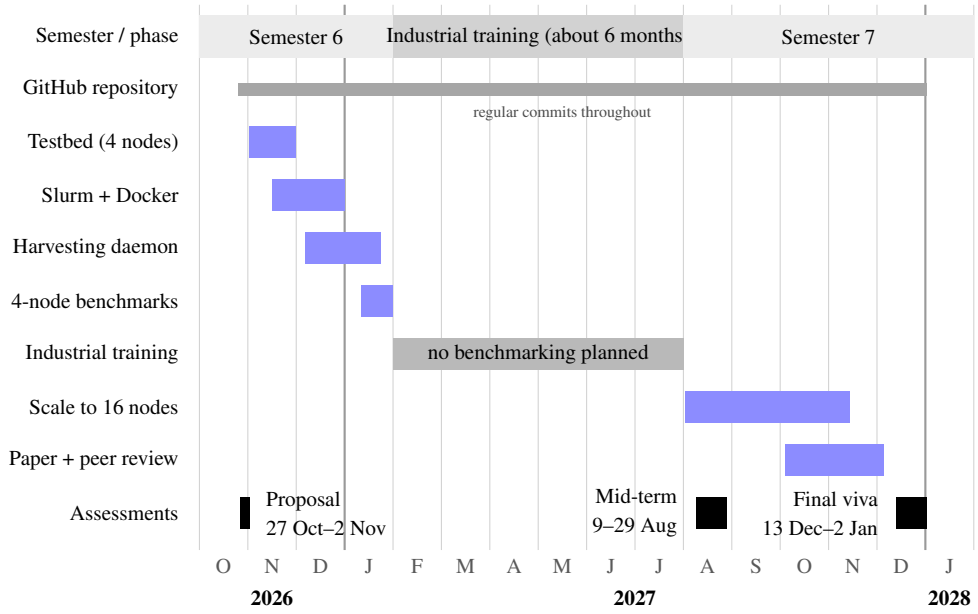


Figure 4: Project schedule by calendar month. Semester 6 and training dates are approximate; assessment dates are from the module guidelines.

9. EXPECTED OUTCOMES

- **Ansible playbooks** that configure networking, Slurm, Docker and NFS on lab workstations.
- **Idle-harvesting daemon** that enlists and evicts nodes automatically, with a measured eviction-latency distribution.
- **Containerized benchmark images** with HPL and the NPB kernels.
- **Scalability results** for $P = 1$ to 16: speedup, efficiency, Karp–Flatt serial fraction, Docker overhead, and the point at which Gigabit Ethernet limits scaling for each workload.
- **Harvesting evaluation:** throughput of independent tasks per night with and without the daemon, and measured lab idle time.
- **Practical benefit:** a no-cost compute resource for off-peak projects, and a base for future work such as checkpointing and GPU containers.

10. CONCLUSION

The idea behind this project is to convert unused laboratory computers into a low-cost HPC cluster using Slurm, Docker and an idle-detection daemon. A 4-node prototype will be built in Semester 6, and in Semester 7 the cluster will be scaled to 16 nodes and tested with HPL and NPB. The results will show how far applications can scale on standard Ethernet hardware and what it costs to evict MPI jobs when a node is reclaimed. Known limitations are that tightly coupled jobs restart from the beginning when a node is reclaimed, and that the head node is a single point of failure. Future work includes checkpointing, rootless container runtimes and GPU support.

REFERENCES

[1] M. Litzkow, M. Livny, and M. Mutka, “Condor—A hunter of idle workstations,” in *Proc. 8th Int. Conf. on Distributed Computing Systems (ICDCS)*, 1988, pp. 104–111.

[2] A. B. Yoo, M. A. Jette, and M. Grondona, “SLURM: Simple Linux Utility for Resource Management,” in *Job Scheduling Strategies for Parallel Processing*, LNCS vol. 2862, Springer, 2003, pp. 44–60.

[3] E. Gabriel *et al.*, “Open MPI: Goals, concept, and design of a next generation MPI implementation,” in *Recent Advances in PVM and MPI*, LNCS vol. 3241, Springer, 2004, pp. 97–104.

[4] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, 2014.

[5] J. J. Dongarra, P. Luszczek, and A. Petitet, “The LINPACK benchmark: Past, present and future,” *Concurrency and Computation: Practice and Experience*, vol. 15, no. 9, pp. 803–820, 2003.

[6] D. H. Bailey *et al.*, “The NAS Parallel Benchmarks,” *Int. J. Supercomputer Applications*, vol. 5, no. 3, pp. 63–73, 1991.

[7] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.